

Relatório para os Membros

Sistema de Folha de Pagamento — UC Dual · Oracle Academy

Análise e Desenvolvimento de Sistemas | Algoritmos e Programação

1. O que é esse sistema?

O projeto é um **Sistema de Folha de Pagamento via terminal (console)**, desenvolvido pensando em pequenas empresas que não têm como investir em soluções caras — seja por assinatura mensal ou anual. A proposta é simples: funcionar sem banco de dados externo, sem dependências e sem internet. Tudo o que o sistema precisa fica local.

Três perfis de funcionário são gerenciados, conforme definição da UC Dual. Os salários são calculados automaticamente, e os dados são persistidos e exportados em CSV — um formato que qualquer planilha (Excel, LibreOffice) consegue abrir.

A arquitetura foi organizada em camadas bem definidas para separar a interface, as regras de negócio e a persistência. O projeto brinca bastante com pacotes em Java — não como maquiagem, mas porque cada camada faz exatamente o que o nome dela diz, e nada além disso.

2. Estrutura de pastas e pacotes

Antes de entrar nas pastas, vale mencionar o **README.md** na raiz: ele é a vitrine do projeto no GitHub — explica o sistema por completo, o motivo de cada decisão e como executar em diferentes ambientes.

Dentro da pasta **sistemaFolha-final**, a estrutura é:

Camada / Pasta	Função
main/	Ponto de entrada — responsável por ligar o projeto todo.
model/	Onde os dados dos funcionários são modelados. Novos tipos de funcionário podem ser adicionados aqui sem mexer no resto.
repository/	Onde os dados são armazenados e recuperados — leitura e escrita em CSV.

service/	Um "fiscal" para serviços mais importantes. Separado para que novas funções possam ser incluídas sem bagunçar outras áreas. O FolhaService conecta todas as camadas.
ui/	A mais trabalhosa: cuida de tudo que o usuário vê no terminal.
docs/	Contém o fluxograma em HTML do funcionamento completo do sistema.

Cada pasta é um **pacote Java** (seguindo a convenção *br.com.folha.[camada]*). Não são simples diretórios: pacotes permitem dividir o sistema em partes confortavelmente isoladas, e ficam declarados sempre no topo de cada arquivo *.java*. Logo abaixo vêm os *imports*, que localizam classes em outros pacotes.

3. Arquivos auxiliares

.gitignore

Diz ao Git o que ignorar. O Git é bastante 'fofoqueiro' — sem esse arquivo, ele tentaria compartilhar tudo, inclusive arquivos temporários e dados sensíveis de funcionários.

COMO_EXECUTAR.txt

Ensina a rodar o sistema de um jeito direto, sem o volume de informações do README. É o guia rápido.

executar.bat

Arquivo de execução com um clique duplo (Windows). Compila e abre o sistema automaticamente, sem precisar abrir terminal manualmente.

Sobre o **.bat**: o arquivo usa três formas do comando *echo* para montar a interface no terminal.

Comando	O que faz
<code>echo Mensagem</code>	Exibe o texto diretamente no terminal.
<code>echo.</code>	Pula uma linha — essencial para a interface não ficar embolada.
<code>@echo off</code>	Impede que o Windows mostre o caminho da pasta antes de cada comando, deixando a tela limpa.

4. O fluxograma em HTML

Dentro da pasta **docs/** há um fluxograma que passa uma ideia geral do funcionamento do projeto. Ele está em HTML por uma razão prática: HTML abre em qualquer dispositivo com navegador, sem precisar de servidor ou instalação. Basta baixar e clicar duas vezes.

Formatos de imagem como PNG e JPEG não preservam detalhes em diagramas de texto. SVG renderizado pode ficar pesado. O HTML resolve os dois problemas — e é universal.

O fluxograma usa bastante estruturas condicionais (principalmente no menu principal) e estruturas de repetição para o loop de algumas funções.

5. Como o código está organizado internamente

Cada arquivo `.java` segue uma estrutura padrão:

```
package br.com.folha.[camada]; ← pacote sempre no topo
```

```
import br.com.folha.[outra]; ← localiza classes em outros pacotes
```

O projeto usa vários tipos de variáveis. Os mais recorrentes são:

Tipo	O que guarda	Exemplo
String	Textos	<code>String nome = "Java";</code>
double	Números com vírgula	<code>double preco = 19.90;</code>
boolean	Verdadeiro ou Falso	<code>boolean ativo = true;</code>
char	Um único caractere	<code>char inicial = 'J';</code>

Classes podem ser **públicas** (acessíveis por outros pacotes), **privadas** (uso interno apenas) ou **abstratas** (servem de molde para outras). Cada `.java` precisa ter pelo menos uma classe pública para que o pacote possa ser acessado de fora.

Orientação a objetos — especialmente **herança** e **polimorfismo** — é o coração da camada *model/*. Esses conceitos ainda estão sendo construídos; algumas partes contaram com auxílio de IA para validar as decisões de design, não para gerá-las do zero.

6. O papel dos membros

Um projeto não é feito só de código. Confesso que os testes não foram muitos — principalmente na interface. Backend não tem a facilidade do frontend de ver o que o código está mudando ao vivo: foi bastante tempo encarando o terminal, e isso é exatamente o tipo de coisa que pode dar problema nas empresas.

Hoje em dia, o que mais existe nas empresas são os **QAs** — profissionais que olham o código do backend e o testam até encontrar quebras. No frontend, quem cumpre esse papel na maioria das vezes é o próprio cliente. E qualquer inconsistência não é apenas uma quebra no código: é uma quebra de credibilidade.

Por isso, além de contribuírem com ideias ao longo do processo, os membros serão os testadores do sistema. Não há como ter certeza do limite de um código sem alguém de fora tentando quebrá-lo — e essa é, portanto, a parte mais importante.

Bons programadores são aqueles que sabem o que estão fazendo e o que isso resulta. Como foi ensinado na Unidade 8: ética no código não é percebida ao programar, mas sim ao testar.

7. Como executar o sistema (resumo)

A forma mais simples é pelo **executar.bat** (Windows): basta ter o JDK 17+ instalado e dar dois cliques no arquivo. O sistema compila e abre automaticamente.

Para quem prefere IDE (VS Code, IntelliJ, Eclipse): abrir a pasta do projeto, navegar até *src/br/com/folha/main/SistemaFolha.java* e clicar em **Run**.

Detalhes completos — incluindo comandos para terminal no Windows, Linux e macOS — estão no **README.md** e no **COMO_EXECUTAR.txt** dentro do projeto.